WebSocket

1. What is WebSocket?

WebSocket is a communication protocol that creates a **persistent full-duplex connection** between client and server over TCP.

Meaning:

```
None
Client can send anytime

Server can send anytime

Single connection stays open
```

Unlike normal HTTP request-response.

2. Core Idea

HTTP usually works like:

```
None
Request → Response → Connection closes
```

WebSocket works like:

```
Connect once

Keep connection alive

Exchange messages continuously
```

3. Full-Duplex Meaning

Both sides communicate simultaneously.

```
Client → Server

Server → Client

At same time
```

Useful for real-time systems.

4. How Connection Starts

WebSocket begins with HTTP handshake.

Client sends:

```
Upgrade request

HTTP → WebSocket
```

Server accepts, then protocol switches.

After that, communication uses WebSocket frames.

5. Example Flow

```
Browser opens chat app

↓

Connects to WebSocket server

↓

User sends message

↓

Server instantly pushes message to others
```

6. Why WebSocket Exists

HTTP polling causes:

```
Repeated requests

Higher latency

More bandwidth waste

Extra server load
```

WebSocket solves this with persistent connection.

7. Best Use Cases

Use WebSocket for:

```
Chat apps

Live notifications

Online gaming

Stock prices

Live sports scores

Collaborative editing

Ride tracking

Video signaling
```

8. Chat Example

```
None
User A sends: Hello

↓

Server pushes instantly to User B

↓

No page refresh needed
```

9. Live Price Example

```
None
BTC = 85000

BTC = 85010

BTC = 84995
```

Server streams updates continuously.

10. WebSocket vs HTTP

Feature	WebSocket	HTTP

Connection	Persistent	New request usually
Communication	Two-way	Mostly request-response
Real-time	Excellent	Limited
Overhead	Low after connect	Higher repeated headers
Simplicity	Medium	High

11. WebSocket vs REST

REST:

```
None
GET /messages

POST /message
```

WebSocket:

```
None
send("hello")

receive("new message")
```

REST for CRUD. WebSocket for real-time streams.

12. Why Faster for Real-Time

```
No repeated handshakes

No repeated headers

Instant push from server

Low latency
```

13. Stateful Nature

Each client connection stays attached to server.

That means server stores:

```
Socket session

Connection metadata

Auth state

Subscriptions
```

Unlike stateless REST.

14. Scalability Challenges

Because connections stay open:

```
None
More memory usage

More open sockets

Sticky routing often needed

Harder load balancing

Connection failover complexity
```

15. Sticky Sessions

Sometimes user must reconnect to same server holding connection state.

Used in:

```
None
Load balancers

Session-aware routing
```

16. Typical Architecture

```
None
Client Apps
   ↓
Load Balancer
   ↓
WebSocket Servers
   ↓
Redis Pub/Sub / Kafka
   ↓
Backend Services
```

17. Why Redis/Kafka Used

If user A connected to Server 1 and user B to Server 2:

```
None
Need message sharing between servers
```

Use:

```
None
Redis Pub/Sub

Kafka

Message bus
```

18. Security

Need:

```
None
WSS (TLS encrypted WebSocket)

Authentication token

Connection rate limits

Origin checks

Idle timeout
```

Use `wss://` instead of `ws://`

19. When Not Ideal

Avoid WebSocket for:

```
None
Simple CRUD APIs

Static websites

Occasional requests

Public APIs needing caching

Low traffic dashboards updated hourly
```

REST simpler there.

20. Browser Example

JavaScript:

```
None
const socket = new WebSocket("wss://api.app.com");

socket.send("hello");

socket.onmessage = ...
```

21. WebSocket vs SSE

Feature	WebSocket	SSE
Client → Server	Yes	Limited
Server → Client	Yes	Yes
Full duplex	Yes	No
Simpler	No	Yes

22. Interview Talking Points

Say WebSocket when system needs:

```
None
Instant updates

Bi-directional messaging

Thousands of live users

Low latency

Real-time collaboration
```

23. Real System Examples

```
None
```

```
WhatsApp Web

Trading dashboards

Google Docs collaboration

Uber live driver tracking

Multiplayer games
```

24. Tradeoffs Summary

Advantages:

```
None
```

```
Real-time

Low latency

Efficient after connection

Bidirectional
```

Disadvantages:

```
None
Stateful

Harder scaling

Connection management

More server resources
```

25. One-Line Summary

WebSocket is a persistent full-duplex communication protocol that keeps a TCP connection open so clients and servers can exchange real-time messages instantly in both directions.